

Your Spending Needs Attention: Modeling Financial Habits with Transformers

D. T. Braithwaite*, Misael Cavalcanti*, R. Austin McEver*, Hiroto Udagawa, Daniel Silva,
Rohan Ramanath, Felipe Meneses, Arissa Yoshida, Evan Wingert, Matheus Ramos, Brian Zanfelicce,
Aman Gupta
Nubank

{daniel.braithwaite,misael.cavalcanti,austin.mcever}@nubank.com.br

Abstract

Predictive models play a crucial role in the financial industry, enabling risk prediction, fraud detection, and personalized recommendations, where slight changes in core model performance can result in billions of dollars in revenue or losses. While financial institutions have access to enormous amounts of user data (e.g., bank transactions, in-app events, and customer support logs), leveraging this data effectively remains challenging due to its complexity and scale. Thus, in many financial institutions, most production models follow traditional machine learning (ML) approaches by converting unstructured data into manually engineered tabular features. Conversely, other domains (e.g., natural language processing) have effectively utilized self-supervised learning (SSL) to learn rich representations from raw data, removing the need for manual feature extraction. In this paper, we investigate using transformer-based representation learning models for transaction data, hypothesizing that these models, trained on massive data, can provide a novel and powerful approach to understanding customer behavior. We propose a **new method enabling the use of SSL with transaction data by adapting transformer-based models to handle both textual and structured attributes**. Our approach, denoted *nuFormer*, includes an end-to-end fine-tuning method that integrates user embeddings with existing tabular features. Our experiments demonstrate improvements for large-scale recommendation problems at Nubank. Notably, **these gains are achieved solely through enhanced representation learning rather than incorporating new data sources**.

1 Introduction

Predictive models form the underpinnings of many systems at financial institutions, such as risk prediction, product recommendations, and fraud detection. Most digital banking platforms have access to large amounts of user data, including **financial transactions, in-app events, and customer support chat logs**. Combined, these sources give us a rich description of what our members need from their trusted financial institution. However, historically, these data sources have been used to extract useful but relatively simple, interpretable features to solve the aforementioned predictive tasks. In this paper, we propose developing transformer-based representation learning models for financial data, specifically financial transactions across a bank’s various offerings (e.g., credit cards, debit cards, transfers). These models facilitate the automated discovery of general features from transactions and can be finetuned to optimize performance across the many applications within Nubank.

Traditional machine learning (ML) models, built for tabular features (e.g., numerical, categorical), have become the norm for industry machine learning systems due to their simplicity and interpretability. Although these approaches work well, designing the numerical and categorical tabular features is labor intensive and requires substantial trial and error. Moreover, such an approach often results in suboptimal information encoding and overlooks text data due to the difficulty of encoding text as tabular features.

In many domains, ML has advanced toward learning representations directly from the raw data for supervised learning tasks. One typical example is convolutional neural networks, which automatically learn features such as edges, textures, and shapes from raw images [23, 35]. Similar approaches are employed by companies such as YouTube [8], Pinterest [27], Meta [31], and Alibaba [24, 28] to learn powerful representations directly from their respective event domains. While these more advanced techniques exist in other domains, most financial industry applications of ML have lagged behind despite the potential for rich behavioral insights to be uncovered from a customer’s financial history.

Another example of the shift towards learning features directly from the data is the use of foundation models, which represent one of the most significant trends in ML. Foundation models have shown the ability to learn generic representations across many domains, such as computer vision [3, 29], audio [2, 11, 30, 41, 43], and natural language [1, 6, 38]. Importantly, the embeddings produced by foundation models typically perform well across a diverse range of tasks. These models are trained on massive amounts of unlabeled data and leverage self-supervised learning (SSL), which involves constructing pseudo labels from the data, e.g., predicting the next words in a sentence. **SSL enables foundation models to learn informative representations of the inputs, thereby eliminating the need for manual feature engineering**. These representations can then solve diverse downstream tasks, relying on the same base model. This is in contrast to manually constructed features, which are often problem-dependent.

As discussed, automatically learning features from transaction data has been relatively unexplored. One recent work by [4] proposes contrastive learning for event sequences (CoLEs) with self-supervision for learning embeddings of user transaction sequences. A subsequent work by [36] proposes an autoregressive next-event prediction approach called NPPR. While both CoLEs and NPPR facilitate the automatic discovery of features from raw transaction data, this paper addresses three limitations of these studies, which are discussed in the following sections.

The first limitation of NPPR and CoLEs is that they only utilize categorical or numerical features from the event sequence. On the

*These authors contributed equally and are listed alphabetically by last name.

other hand, we hypothesize that **there is much value in learning from natural language features, e.g., descriptions**. Secondly, both use RNN models, which afford efficiencies in real-time inference settings. However, RNNs are not as effective as Transformers [39] for modeling long-range relationships in the input. This is especially important for transaction data, which has seasonal patterns. Finally, the scale of data in these studies (billions of transactions) is orders of magnitude smaller than what we work with at Nubank (O(100B) of transactions across 100M+ members). This is important because we know that scaling such models can result in emergent properties. For example, large language models learn how to answer questions or summarize text simply by observing natural language [6].

In this paper, we propose a novel formulation of transaction data that facilitates transformer-based representational models. Our proposed setup is based on the ‘text-is-all-you-need’ approach of [25], except that it utilizes additional special tokens to reduce the context length. This reduces the problem of transaction modeling to something that can be solved using standard self-supervised losses, like next-token prediction. Moreover, this formulation enables the modeling of arbitrary string, numerical, and categorical attributes, whereas previous works have focused primarily on tabular features. Following the development of our representation learning models for transaction data, we propose an **end-to-end finetuning approach, allowing us to blend user embeddings with existing tabular features**. Finally, we demonstrate that these finetuned models can substantially improve performance on large-scale problems at Nubank with a specific application to recommendation. Importantly, **these gains are achieved without incorporating any new sources of data; instead, they are achieved by allowing the transformer to learn a more advanced representation of the transaction data, originally represented as handcrafted features**.

2 Related Work

Sequential Recommendation. Sequential recommendation (SR) systems involve modeling a user’s behavior from a sequence of item interactions (e.g., purchases, clicks, etc) and attempting to predict future actions to recommend these to the user. This is closely related to modeling transaction sequences, as items also consist of a rich set of attributes, such as text, numerical, categorical, and images.

One of the earliest applications of transformer models to SR is SASRec [20], which utilizes an ID-based representation of items. In this setup, each item is assigned a unique ID, and item embeddings are obtained from an embedding table. Importantly, item attributes are not utilized. A causal transformer is then trained using a generative next-item prediction objective, which is a self-supervised loss. This approach outperformed other state-of-the-art models on standard SR tasks. However, ID-based approaches suffer from the so-called cold-start problem, where it is not possible to generalize to items unseen during training.

[37] builds on the result of SASRec, using the same ID-based approach. The authors propose the BERT4Rec model, which uses a bi-directional attention mechanism rather than a causal one. Hence, they also use a masked language modeling task. The authors show that using this alternative model consistently improves performance across many baselines.

More recent works have looked to alleviate the cold start problem by using language models (LMs) to embed items. For example, [13] proposes a model called ZESRec, which uses a pre-trained BERT [12] model to generate item embeddings. The authors then use a Bayesian approach to model the user sequences of item interactions. The trained model can outperform existing methods and generalize across domains. Another related paper proposes the UniSRec [17] model, which learns item embeddings by applying a parameterized whitening procedure to BERT embeddings. Finally, [16] extends the BERT4Rec model to use LLM (i.e., large models) based item embeddings as input. This straightforward change results in 15-20% improvement on benchmark tasks.

The models discussed thus far use LM embeddings to improve the ability of product recommendation systems to generalize to unseen items and domains. However, these LMs are pre-trained on general natural language rather than the specific data used in each task. [25] proposes formulating the SR problem as a text problem. Their approach, denoted the *Recformer*, constructs item sentences by flattening the key-value pairs into a string and treating everything as text. This model is then trained using the standard masked language modeling objective, along with an additional contrastive task designed to enhance item representations. The Recformer outperforms all other approaches discussed in this section thus far.

Finally, [27] and [31] propose PinnerFormer and NxtPost, respectively, for delivering content recommendations at Pinterest and Meta. In both cases, the recommended items are complex and can comprise images, text, and tabular features. Both papers utilize an event encoding model to embed pins or posts into a latent space, followed by the training of a causal transformer model to predict future behavior. Both of these papers are good examples of how these models can be extended to multimodal domains.

Sequential Modeling for Financial Data. In what follows, we explore applications of sequential modeling systems to financial transactions. Contrastive Learning for Event Sequences (CoLES) [4] is a contrastive approach for learning embeddings of user event sequences. It uses an RNN-based encoder to embed subsequences from the same user and contrast these against sequences from other users. More specifically, embeddings for subsequences from the same user are driven closer together, whereas embeddings corresponding to different users are driven further apart. The authors evaluated their method by finetuning their pre-trained user embeddings on both public and large-scale proprietary banking datasets. **By comparing their performance against models using traditional hand-crafted features and other sequence-based methods, they successfully demonstrated that the CoLES embeddings provided a significant and consistent performance uplift.**

NPPR [36] is an auto-regressively trained RNN model using a self-supervised next item prediction task. They also optimize the current embedding to predict past behavior. Importantly, they demonstrate the success of using generative tasks, such as next-item prediction, for learning event sequence embeddings. NPPR also only utilizes numerical or categorical features from the events. On the same benchmarks as CoLES, NPPR achieved state-of-the-art results.

3 A Transformer-Based Model for Transactions

Our goal is to ingest a member’s time-ordered transactions and represent their financial behavior as an embedding. **Each transaction is represented by text along with numerical and categorical attributes.** As is common in other domains like natural language, images, and audio, we show that it is possible to efficiently summarize member behavior by learning to predict their future transactions.

In this section, we introduce our foundation model formulation based on the transformer [39]. We choose a causal transformer architecture (GPT-like), as opposed to a BERT-like model or an RNN, for several reasons. First, the state-of-the-art industry approaches to sequential recommendation use a transformer backbone [27, 31]. Secondly, transformers offer computational advantages over RNNs [19, 32] during training and inference time, since we are not performing autoregressive generation. Finally, they support long range dependencies between inputs. This is especially useful for transaction data because spending money has seasonal variation (e.g. people might spend more money in the holiday season).

In what follows, we outline the structure of this section: Firstly, in section 3.1, we introduce a modified version of the text-is-all-you-need approach of [25] as our interface between transactions and transformers. Following this, in section 3.2 we discuss a supervised finetuning setup to allow tuning embeddings for specific tasks. Finally, in section 3.3 we present an extension of the model, denoted joint fusion, that facilitates end-to-end finetuning with additional tabular features. This is especially important since there are often critical features that are tabular by nature. We denote our joint fusion model *nuFormer*

3.1 Transaction Transformer Formulation

In this section, we begin by introducing our approach for converting a user, which consists of thousands of transactions, into something that transformer [39] based sequence-to-sequence models can process (i.e., a sequence of embeddings). Formally, we define a **transaction** as a collection of key-value pairs $t = \{(k_1, v_1), \dots, (k_m, v_m)\}$. Then, a member consists of a sequence of transactions:

$$u_i = \{t_1^{(u_i)}, \dots, t_{N_{u_i}}^{(u_i)}\}$$

For the purposes of this paper, we will assume a transaction consists of **three attributes: the amount represented as a floating point number, the date represented as a timestamp, and finally, a description represented as a string.** Hence, in this case each user transaction is defined as:

$$t = \{(amt, v_{amt}), (date, v_{date}), (desc, v_{desc})\}$$

While this setup is simplistic, we can build representations for the many unique attributes of transactions, such as merchant ID, location, status, merchant category, number of installments, etc.

As discussed, we must first construct an interface between transactions and transformers. One option is to assign IDs to transactions as done in sequential recommendation literature, like SASRec [20]. However, this faces challenges such as the variability of transaction descriptions over time, leading to a large ID space, and the cold start problem for unseen transactions. Alternatively, we could adopt a text-is-all-you-need [25] approach, converting transaction attributes into a JSON string treated as natural language, which

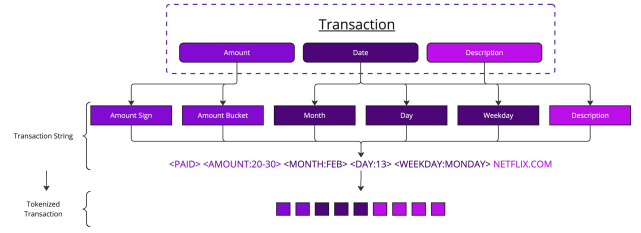


Figure 1: This figure shows the process of converting a transaction into a stringified and then tokenized form. The amount and date fields are converted into the corresponding special tokens and these are concatenated with the description. After tokenization, each special token is represented by one token, whereas the natural language description is represented by potentially many tokens.

helps in generalizing to unseen transactions. The downside of this method is the large number of tokens it generates per transaction, raising concerns about the quadratic scaling of attention operation costs with context length.

For this initial modeling approach, we chose a modified version of text-is-all-you-need. Specifically, we represent numerical and categorical features using special tokens (numerical features are first quantized into bins to make them categorical). Formally, we define a tokenizer as having a vocabulary $\mathcal{V}$ size of $|\mathcal{V}| = V$ tokens. This vocabulary contains a subset of predefined **special tokens** $\mathcal{V}_S$, with $|\mathcal{V}_S| = V_S$. The remaining tokens, $\mathcal{V}_{\text{text}} = \mathcal{V} - \mathcal{V}_S$, are for general text and constructed using an algorithm like **byte-pair-encoding (BPE)** [14, 33]. The special token set is crafted from the specific attributes we want to model, specifically the amount, date and description. Specifically, we have the following feature to token mappings:

- Amount Sign ($\phi_{\text{sign}} : \mathbb{R} \rightarrow \mathcal{V}_{\text{sign}}$, where $|\mathcal{V}_{\text{sign}}| = 2$): A separate token to represent whether the transaction amount is an inflow or outflow.
- Amount Bucket ($\phi_{\text{amt}} : \mathbb{R} \rightarrow \mathcal{V}_{\text{amt}}$, where $|\mathcal{V}_{\text{amt}}| = 21$): The amounts are binned and a separate token is assigned to each bin.
- Date Features: are all represented with their own tokens.
 - Month ($\phi_{\text{month}} : \mathbb{R} \rightarrow \mathcal{V}_{\text{month}}$, where $|\mathcal{V}_{\text{month}}| = 12$): One of the 12 possible months of the year.
 - Day ($\phi_{\text{day}} : \mathbb{R} \rightarrow \mathcal{V}_{\text{day}}$, where $|\mathcal{V}_{\text{day}}| = 31$): One of the possible 31 days.
 - Weekday ($\phi_{\text{weekday}} : \mathbb{R} \rightarrow \mathcal{V}_{\text{weekday}}$, where $|\mathcal{V}_{\text{weekday}}| = 7$): One of the possible 7 week days.
- Text Description ($\phi_{\text{BPE}} : \text{str} \rightarrow (\tau_1, \dots), \tau_i \in \mathcal{V}_{\text{text}}$: Tokenized as natural language using a standard tokenizer.

where $\mathcal{V}_S = \mathcal{V}_{\text{sign}} \cup \mathcal{V}_{\text{amt}} \cup \mathcal{V}_{\text{month}} \cup \mathcal{V}_{\text{day}} \cup \mathcal{V}_{\text{weekday}}$. Hence, we can define a tokenized transaction as:

$$\tau(t) = (\phi_{\text{sign}}(v_{\text{amt}}), \phi_{\text{amt}}(v_{\text{amt}}), \phi_{\text{month}}(v_{\text{date}}), \phi_{\text{day}}(v_{\text{date}}), \phi_{\text{weekday}}(v_{\text{date}})) \oplus \phi_{\text{BPE}}(v_{\text{desc}}) \quad (1)$$

How do the tokens are generated?

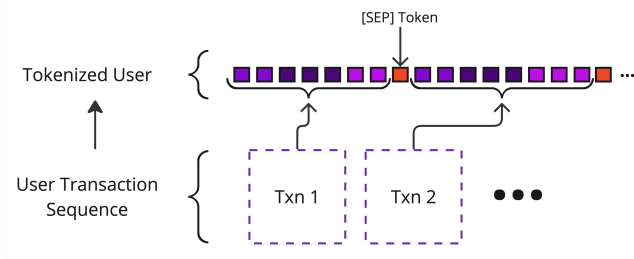


Figure 2: This figure shows the process of constructing a tokenized member. This process involves taking each transaction and concatenating their string representations, while inserting separator tokens in-between each transaction.

where $\oplus$ denotes vector concatenation. An example of (1) is shown in figure 1. Next we can extend this technique to tokenize a member's account, given by $u_i = \{t_1^{(u_i)}, \dots, t_N^{(u_i)}\}$, by concatenating the transaction strings with intermediate separator tokens, $\tau_{sep} \in \mathcal{V}_{text}$:

$$\mathbf{x}_i = \left(\bigoplus_{j=1}^{N_i-1} (\tau(t_{i,j}) \oplus (\tau_{sep})) \right) \oplus \tau(t_{i,N_i}). \quad (2)$$

Figure 2 shows this process.

Finally, we pre-train causal transformers on our tokenized user representation using the standard next token prediction (NTP) task, which involves classifying the next token to occur out of all possible tokens. This choice is due to the success of standard language modeling tasks in the text-is-all-you-need approach [25]. As with transformers trained on natural language, transaction tokens are embedded using a lookup table. Figure 3 shows this structure.

Since the attention operation is invariant to a given token's position in the sequence, it is common to add an encoding of each token's position to the input of the model. [21] showed that causal models learn their own form of position representation. The position information induced by training with a causal mask also generalises to longer sequences during inference [21]. Hence, we choose to use no positional embeddings (NoPE) [21]. Finally, we also use FlashAttention [9, 10, 34]. Both NoPE and FlashAttention allow us to train on large context lengths within a single A100 80GB GPU. In practice, we often train on clusters of H100 or H200 GPUs using distributed data parallel or fully sharded training.

3.2 Finetuning Embedding Models

In the previous section, we introduced a formulation of transaction sequences as natural language. This facilitated pre-training user embedding models on transaction data, and learning general embeddings of user behavior. In practice, however, such models are often fine-tuned for specific tasks to achieve state-of-the-art performance. While in this section, we consider finetuning our models for binary classification problems, the same technique can be used for multi-class or even regression problems.

During supervised finetuning, a member consists of a sequence of transactions: $u_i = \{t_1^{(u_i)}, \dots, t_{N_{u_i}}^{(u_i)}\}$, and a label l_i . We wish to learn a function $f_{\theta}(u_i) = l_i$. To achieve this, we take the final token (before any padding) embedding as the user representation, and

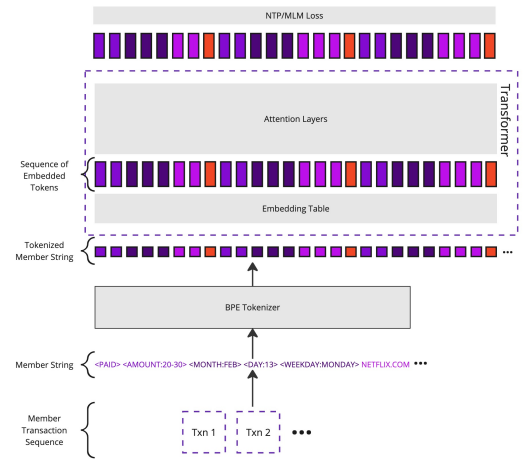


Figure 3: Modeling member sequences starts with the member's transaction sequence, and first constructs the members string representation by concatenating the transaction strings. A tokenizer is applied to get the tokenized member string. The sequence of embedded tokens is obtained using a learned embedding table, and forms the input to the attention layers. Finally, we have the next token prediction loss computation used to train the model.

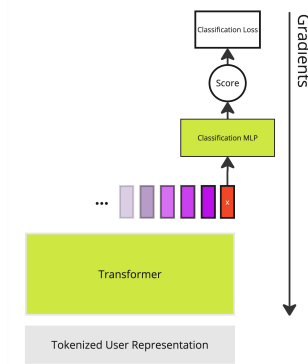


Figure 4: This figure shows how a pre-trained transaction foundation model is augmented for finetuning (green indicates trainable parameters). We add an MLP which produces a score from the final output embedding. Both, the MLP and transformer are optimized to minimize the classification loss.

add an MLP network to the model, which reduces this embedding to a score. We choose the final token embedding because, since the model is causal, it is the only token with the full context of the user. Then, we optimize the MLP and the transformer weights to minimize the cross-entropy error. This is shown in figure 4.

In preliminary experiments, we found that finetuning the entire transformer often leads to overfitting and catastrophic forgetting.

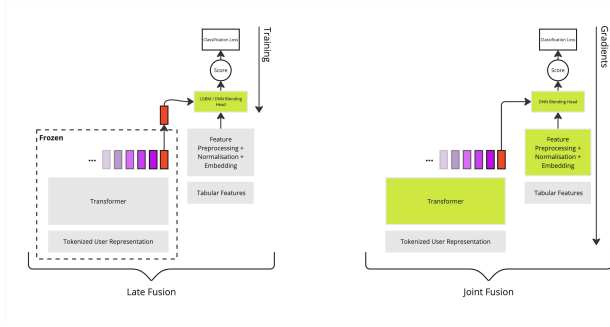


Figure 5: This figure shows the difference between late fusion and joint fusion (green boxes indicate learnable parameters). For late fusion, frozen embeddings are taken from the foundation model, and only the classifier model is trained. In the case of joint fusion, the classification network is trained simultaneously with the transformer layers.

Hence, we use LoRA [18] to help prevent these issues. This finetuning approach allows us to deliver substantial improvements over the unsupervised embeddings on a diverse range of problems.

3.3 Modeling Tabular Features with DNNs

A primary motivation for developing transaction-based user embedding models is to alleviate the need for manual feature engineering. In the previous section, we saw how to refine these features for specific tasks through finetuning. However, in many cases, there are either hand-crafted features from non-transaction sources or features that are tabular by nature that are critical to model quality. Since these handcrafted features are orthogonal to the transaction data used to train the embedding models, we need to incorporate them into the final prediction. **This process of combining embeddings with tabular features is denoted *blending* or *fusion*.**

Gradient-boosted tree (GBT) models are generally considered state-of-the-art for dealing with tabular data [5], e.g., XGBoost [7] or LightGBM [22]. There are two common approaches for blending with GBTs. The first is early fusion, which blends pre-trained embeddings with tabular features using GBTs. The second is late fusion, a two-stage process, where we first finetune the transformer on a subset of the data and then train the GBT model to combine the finetuned embeddings with tabular features.

Late fusion allows for embeddings that are tailored to the task of interest and performs better than early fusion. However, these finetuned embeddings are learned in isolation from the features. We hypothesize that finetuning jointly with blending will allow the transformer to better capture interactions between the tabular features and embedded transaction data. Hence, in contrast to both early/late fusion, we propose a system that can be trained end-to-end, allowing the model to learn an optimal blending of tabular and sequential data, which we denote as joint fusion. Figure 5 shows the joint and late fusion systems.

GBTs are not compatible with joint fusion because there are no gradients from the GBT to propagate to the transformer. Motivated by these observations, we invested in DNN-based tabular feature networks, for which recent works have started to show they can

be competitive with GBTs [42]. However, the performance of DNN tabular feature networks can vary drastically between problems. For example, one survey paper [26] evaluated 19 different tabular feature models (NN + GBT) on 176 distinct datasets. The authors found that each of the 19 different models outperformed all others on at least one dataset. On the other hand, there also existed at least one dataset where each model simultaneously performed the worst. Hence, making it challenging to have a one-size-fits-all approach. Therefore, we need to find a configuration that works for our problems.

The first step in our approach was to achieve parity between DNNs and GBT models on only the tabular features. We selected the DCNv2 architecture [40] as it has shown success on related problems at a large scale (e.g., used by Google). However, initial results showed much worse performance for the DNN-based DCNv2 models than for the GBTs.

The recent paper [15] found that by embedding numerical attributes, they achieved significant gains when modeling numerical tabular features in DNNs. These numerical embeddings are constructed using periodic activations at different (learned) frequencies. We combined this with trainable embedding tables to also facilitate categorical feature embeddings. In the next section, we show that incorporating this embedding strategy into the DCNv2 model allowed us to achieve parity with GBTs on our cross-sell problem.

Despite achieving parity with only tabular features, the last challenge to overcome was incorporating embeddings into these models while maintaining or beating the GBT model’s performance with DNNs. Three key factors were critical in achieving this. First, we implement modifications to the DCNv2 architecture for our use case, utilizing its cross-layer capabilities to process only the embedded tabular features and project the result into a low-dimensional embedding. This feature embedding is concatenated with the transaction embedding (from the transformer), and a multi-layer perceptron is used to make the final prediction. Secondly, adding regularization in the form of weight decay and/or dropout to the customized DCNv2 layers reduced overfitting. Finally, adding normalization to the transaction embeddings improved the consistency of the customized DCNv2, allowing the DNN model to outperform the GBTs reliably. Despite the challenges in using DNNs with tabular data, we achieved a model that works well for our current tasks of interest by combining DCNv2, numerical, and categorical feature embeddings, along with regularization. We denote this joint fusion model as *nuFormer*.

4 Experimental Results

In this section, we empirically evaluate our transformer-based embedding model for transaction data on a practical task at Nubank. First, in section 4.1 we introduce the recsys problem in more detail as well as the LightGBM baseline. Next, in section 4.2 we demonstrate the process of building our DNN tabular feature model that achieves parity with LightGBM. Then, in section 4.3 we explore how pre-training and joint fusion scale as a function of several model properties (model size, context length, data volume). Finally, in section 4.4, we apply our transformer-based embedding models to a practical recsys modeling task using backtest data. Importantly, we show that we can achieve a 1.25% relative improvement in test set

AUC by using our foundation models; this lift in performance is 3x a typical model launch that leads to a significant business outcome. This section includes a discussion of our production deployment of these models at Nubank.

4.1 Recsys Problem and Baseline Setup

In this section, we introduce the recsys problem that we use in following sections to demonstrate the success of our representation learning models. The data consists of 203M training rows and 2M testing rows, where each row corresponds to a particular label and timestamp combination for a given user. Specifically, the same user might occur multiple times in the dataset, but at different times, with potentially different labels and/or transaction sets. The label is binary, where 1 represents a positive user interaction (e.g., activating/using a recommended financial product) and 0 represents no interaction. Importantly, this label is time-delayed, which means we are attempting to predict the user behavior in six months from the score date. The time periods covered by the train and test sets are disjoint.

Each row can contain potentially many transactions, though in some cases, members might have no transaction history. For these experiments, the transactions of a member can be generated from three independent financial products. The specific products don't matter as much for the experiment so we denote them as sources A, B and C. In section 4.3.1, we explore the relative importance of each source.

In total, there are 291 tabular (numerical or categorical) features. Some of these features are derived from transaction data sources (e.g., average spend in a certain period). However, non transaction sources are also included, e.g., bureau scores. The baseline is a LightGBM model trained on this hand-crafted feature set. On the other hand, the challenger is a late or joint fusion model that incorporates the learned transaction embeddings with these tabular features. Of course, such a challenger setup contains redundancy. However, in practice, we can remove most of the hand-crafted transaction features without any loss in performance.

4.2 Tabular Feature Modeling Parity with DNNs

In this section, we explore the aspects of the DNN model that allow us to match the LightGBM performance when modeling the tabular features (numerical and categorical) highlighting the incremental gains obtained towards the challenger DNN model. Table 1 shows the relative improvement over the baseline (LightGBM) as we improve our DNN tabular feature model. It is important to highlight that the results shown here for each DNN and LightGBM model are obtained via hyperparameter tuning.

The first model was a multilayer perceptron architecture (MLP), preceded by a processing step that applies standardization to numerical features and one hot encoding to categorical features. Table 1 shows that the MLP approach was unable to match the performance of LightGBM. The second was the Deep Cross Network V2 (DCNv2) architecture designed by Google [40], which is capable of modeling explicit and implicit interactions of input features to make the final prediction. For this particular network, a different feature processing is employed: numerical features are transformed through a signed *log1p* function and categorical features are mapped

Model	Relative Test AUC Improvement
MLP	- 0.44%
DCNv2	- 0.09%
MLP + PLR	- 0.23%
LightGBM (Baseline)	
DCNv2 + PLR	+ 0.06%
DCNv2 + PLR + L2 Reg	+ 0.08%

Table 1: Relative test AUC improvements over the baseline (features only LightGBM) for different DNN configurations.

to learnable embeddings via lookup table. These transformations were necessary to avoid numerical instability in the cross layers, as they do not work with sparse vectors.

Although DCNv2 achieved higher performance than MLP, it still lagged behind LightGBM baseline. Recent work [15] has shown that representing numerical attributes as dense embeddings helps to improve the performance of neural networks on tabular-based problems, either for MLP or Transformer based architectures. Following that idea, we retrained both MLP and DCNv2 models using the periodic linear (PLR) embedding approach for numerical features, which maps a real number to a dense embedding of parametrized size (number of frequencies), whose elements are periodic activations (sin and cosine) of the numerical value. Results on Table 1 show the effectiveness of this embedding technique: MLP and DCNv2 with numerical periodic embeddings outperform their previous performances, and DCNv2 + periodic embeddings was able to match the performance of LightGBM.

The use of numerical embeddings usually leads to an increase in the number of parameters of DNNs because the input vector becomes wider (depending on the dimension of the embedding). Naturally, the model becomes more prone to overfit and requires regularization to avoid losses in generalization power. Thus, in a following step, we applied L2 regularization to DCNv2 weights, which further increased performance on the test set, furthering the DCNv2 advantage against LightGBM for this problem.

4.3 Exploring Pre-Training and Finetuning of Transformer-Based User Embedding Models

This section presents an analysis of how the embeddings of our finetuned LMs improve in quality as we scale and various other ablations. To begin, we define the **baseline foundation models, of which there are two, with 24M and 330M parameters**, respectively. Both have a context length of 2048 and use all the transaction sources. These **models are pre-trained and finetuned on 20M rows**. The baseline transformers take advantage of description, amount, and date, as described in section 1. Then, in section 4.3.1, we explore different combinations of transaction sources. This section is done within the context of early fusion, due to the volume of experiments. Following this, all experiments use joint fusion and results are reported as the absolute improvement in AUC over a GBT baseline trained only on tabular features using all 203M users. In section 4.3.2 we show that using larger models allows us to learn better features from the raw transaction data. Then, in section 4.3.3, we examine how varying the context length affects the performance

Source Combination	Absolute AUC Change
A	0.72
B	-8.21
C	-20.52
AB	0.91
BC	-12.24
AC	-0.27
ABC	[baseline]

Table 2: Impact of data sources on recsys using GBT . Using individual sources, and their combinations.

of joint fusion. Finally, in section 4.3.4, we explore how the volume of data used for finetuning affects the downstream performance.

4.3.1 Data Source Combinations. Nubank’s users can have multiple types of accounts, cards, and transactions within their event history, which can be broken down into numerous data sources, including credit card, debit card, open finance, wires, transfer activity, and a variety of bill items like credit card payments and fees. Importantly, when using the tokenization procedure described in section 1, each transaction consumes 14 tokens on average. Hence, given that our LMs have a limited capacity and context window, it is important to explore the power and impact of each individual data source to find the blend and balance for optimal performance.

We show that even though every source of data adds a different view of the consumer’s activity, they might not all incrementally add value to the predictive power of the model. To study the effect of each transaction set, we pretrained numerous 24M models with various combinations of 3 anonymized data sources (A, B, C), extracted those pretrained embeddings, and trained a GBT to make predictions based on the information stored in the pretrained embeddings (i.e, early fusion). Table 2 shows the absolute change in the AUC when using each source combination compared to a baseline (uses all the sources). Overall, data source A clearly contains powerful information relevant for recommendation. Transactions from B also contain useful signal that appears to be orthogonal to A, but C may slightly confuse the model or take attention from more pertinent transactions when combined with other sources.

Interestingly, in some cases, adding a data source can cause a decrease in performance. For example, $BC < B$ and $ABC < AB$. This is caused by the additional transactions causing increased contention in the already limited context window. In this case, transactions from B appear to contain the least useful information, and when we include them, it pushes more useful transactions from A and C out of the model’s visibility.

This aforementioned effect can be caused by the difference in information densities and transaction frequencies between sources. In some cases, transaction sources can be sparse but relevant. For example, bill items can contain information about buy-now-pay-later loans. On the other hand, debit card can contain many (frequent) small transactions which are overall less important.

4.3.2 Impact of Model Size. We compare the AUC performance of finetuned models of two different sizes: one model consists of approximately 24M parameters while the other has approximately 330M parameters. We use a causal GPT-like decoder-only model,

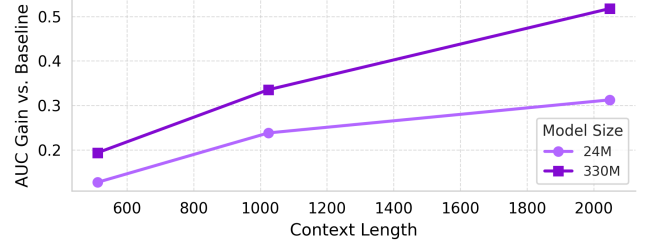


Figure 6: nuFormer test AUC for different context lengths (512, 1024, 2048) with 24M and 330M models.

with 24 attention layers, each with 16 attention heads. Importantly, the two model configurations differ only in the hidden embedding size where the 330M model has a 1024 length embedding and the 24M model has a smaller 256 length embedding.

# paramaters	Absolute AUC Gain
24M	0.3123
330M	0.5177

Table 3: This table shows how the nuFormer performance for two different settings of the model size.

Table 3 shows the performance of the baseline model configuration with only the size varied. We clearly see an improvement from the increased capacity. **The findings in this section suggest that there is a rich set of information in our data that larger models can exploit.** In future work, we plan to continue experimenting with scaling up to larger models. More specifically, we plan to derive scaling laws for these transaction based user embedding models.

4.3.3 Effect of Context Length on Joint Fusion. In this section, we explore how varying the context length affects the performance of joint fusion. More specifically, we compare the baseline configuration using context lengths of 512, 1024 and 2048 tokens for both the 24M and 330M variants. Figure 6 shows these results, where we see that **larger context lengths lead to improved performance. Moreover, we see that the larger model is better able to exploit the information in the longer contexts.**

4.3.4 Effect of Training Data Volume for Joint Fusion. In this section, we demonstrate that the performance of our joint fusion scales as a function of the data volume. Specifically, we take two models, one 24M and one 330M, both pre-trained on 20M rows. We then finetune each of these two models on 5M, 20M, 40M, 100M rows, and plot the results, which is shown in figure 7. Importantly, we see a clear advantage to using more data with joint fusion, and that bigger models obtain a larger advantage from more data.

4.4 Modeling Recommendations at Nubank

This section presents a practical application of our transaction data foundation model to a recommendation problem. This problem involves predicting whether a member will activate & use a product when the communication is sent. The baseline for this task attempts to predict this behavior from hand-crafted tabular features using

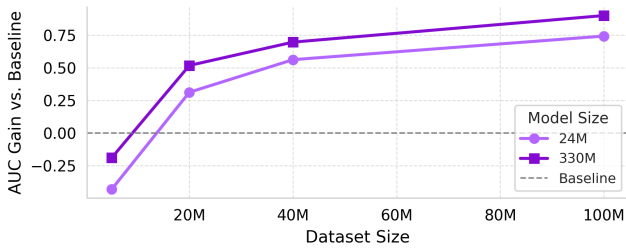


Figure 7: *nuFormer* test AUC for different amounts of training data (5M, 20M, 40M, 100M) with 24M and 330M models.

Model	Relative Test AUC Improvement
Late Fusion (LightGBM)	0.97%
Late Fusion (DCNv2)	0.97%
<i>nuFormer</i>	1.25%

Table 4: Relative test AUC improvements over the baseline for different blending approaches.

LightGBM. Importantly, some of these tabular features are derived from transactions. Hence, our transaction embeddings do not add any new sources of data to the model. Rather, we are allowing the model to learn its own transaction features from the raw data.

We compare several different strategies to clearly demonstrate the advantage of our final Joint Fusion approach, which achieves an overall relative improvement of +1.25% in test AUC over the baseline model trained only on the tabular features. For reference, this is 3x the improvement typically observed when successful models realize material business impact. Furthermore, historical model improvements were obtained from adding new data sources or signals to the model. In our case, this improvement is entirely from advancements in modeling.

The data consists of one record per user interaction. Hence, the total amount of data is much greater than just one row per member. In the following experiments, we want to analyze the relative gain in test AUC from adding the foundation model user embeddings. We blend this model with the tabular features in the following ways:

- (1) Baseline: The existing model uses the handcrafted tabular features and is a LightGBM trained on the entire dataset. This model has no user embeddings.
- (2) Late Fusion: We finetune the model on a 20% subset of the data rows without any features, and then we use the remaining rows to train the downstream models to predict the label from the embeddings and features.
- (3) *nuFormer*: Uses the joint fusion procedure on 203M rows.

Table 4 shows the relative improvements over the baseline in the test AUC from adding transaction based user embeddings.

A major concern was whether the joint fusion based solution would overfit to transaction attributes (e.g., descriptions) from specific time periods. This is especially important because labels take six months to mature, so we cannot train these models on the most recent transactions. During the back-testing, we constructed an extended test set, covering a 6 month period after the data used

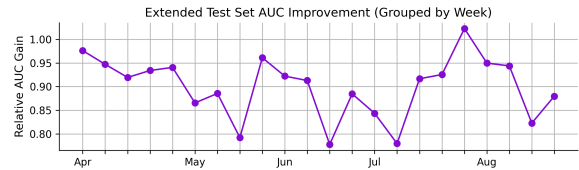


Figure 8: Extended test set (out-of-time) stability analysis. Compares *nuFormer* and the baseline.

during joint fusion. Figure 8 shows the relative difference in AUC for this extended test set. Importantly, we see a consistent gain over the baseline as we get further from the train period.

The Joint Fusion (DCNv2) model was deployed to production with the primary goal of reducing long-term user churn. The success of the system is measured by a reduction in churn events observed 6 months after a user adopts the product. The long lead-up to collecting the label required for the business outcome prevented us from A/B testing every variant. This is a problem where we have historically recorded that the business outcome closely follows improvements in offline AUC. This was one of the most successful models that reduced churn by 4.4% relative to the baseline model. This result highlights the model’s effectiveness in delivering recommendations that foster lasting user engagement.

5 Conclusion

In this paper, we introduced a novel approach to leveraging transformer based embedding models for financial data, transforming raw transactions into actionable insights. While these models build on standard data sources used throughout the industry, they facilitate automatically learning informative features that may be not obvious to data scientists. In an empirical evaluation, we saw how **using joint fusion to tune our transaction foundation models for a practical recsys task could generate substantial lifts**. These foundation models can be leverage for tasks across Nubank, improving Nubank’s ability to understand their consumers so we can help them meet their financial needs at the right time. Moreover, we **saw a clear advantage to scaling the model size, context length, and training data**. In future work, we plan to develop rigorous scaling laws for these models and show that they are powerful foundation models by applying them to a diverse collection of problems.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Junyi Ao, Rui Wang, Long Zhou, Chengyi Wang, Shuo Ren, Yu Wu, Shujie Liu, Tom Ko, Qing Li, Yu Zhang, et al. 2021. Speech5: Unified-modal encoder-decoder pre-training for spoken language processing. *arXiv preprint arXiv:2110.07205* (2021).
- [3] Muhammad Awais, Muzammal Naseer, Salman Khan, Rao Muhammad Anwer, Hisham Cholakkal, Mubarak Shah, Ming-Hsuan Yang, and Fahad Shahbaz Khan. 2023. Foundational Models Defining a New Era in Vision: A Survey and Outlook. *arXiv preprint arXiv:2307.13721* (2023).
- [4] Dmitrii Babaev, Nikita Ovsov, Ivan Kireev, Maria Ivanova, Gleb Gusev, Ivan Nazarov, and Alexander Tuzhilin. 2022. Coles: Contrastive learning for event sequences with self-supervision. In *Proceedings of the 2022 International Conference on Management of Data*. 1190–1199.
- [5] Vadim Borisov, Tobias Leemann, Kathrin Seßler, Johannes Haug, Martin Pawelczyk, and Gjergji Kasneci. 2022. Deep neural networks and tabular data: A survey.

- IEEE transactions on neural networks and learning systems* (2022).
- [6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [7] Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 785–794.
- [8] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.
- [9] Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691* (2023).
- [10] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems* 35 (2022), 16344–16359.
- [11] Nilaksh Das, Saket Dingliwal, Srikanth Ronanki, Rohit Paturi, Zhaocheng Huang, Prashant Mathur, Jie Yuan, Dhanush Bekal, Xing Niu, Sai Muralidhar Jayanthi, et al. 2024. Speechverse: A large-scale generalizable audio language model. *arXiv preprint arXiv:2405.08295* (2024).
- [12] Jacob Devlin. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [13] Hao Ding, Yifei Ma, Anoop Deoras, Yuyang Wang, and Hao Wang. 2021. Zero-shot recommender systems. *arXiv preprint arXiv:2105.08318* (2021).
- [14] Philip Gage. 1994. A new algorithm for data compression. *The C Users Journal* 12, 2 (1994), 23–38.
- [15] Yury Gorishniy, Ivan Rubachev, and Artem Babenko. 2022. On embeddings for numerical features in tabular deep learning. *Advances in Neural Information Processing Systems* 35 (2022), 24991–25004.
- [16] Jesse Harte, Wouter Zorndrager, Panos Louridas, Asterios Katsifodimos, Dietmar Jannach, and Marios Fragkoulis. 2023. Leveraging large language models for sequential recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 1096–1102.
- [17] Yupeng Hou, Shanlei Mu, Wayne Xin Zhao, Yaliang Li, Bolin Ding, and Ji-Rong Wen. 2022. Towards universal sequence representation learning for recommender systems. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 585–593.
- [18] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv* 2021. *arXiv preprint arXiv:2106.09685* (2021).
- [19] MI Jordan. 1986. *Serial order: a parallel distributed processing approach. technical report, june 1985-march 1986*. Technical Report. California Univ., San Diego, La Jolla (USA). Inst. for Cognitive Science.
- [20] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.
- [21] Amirhossein Kazemnejad, Inkrit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. 2023. The impact of positional encoding on length generalization in transformers. *Advances in Neural Information Processing Systems* 36 (2023), 24892–24928.
- [22] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems* 30 (2017).
- [23] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. 2012. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems* 25 (2012).
- [24] Chao Li, Zhiyuan Liu, Mengmeng Wu, Yuchi Xu, Huan Zhao, Pipei Huang, Guoliang Kang, Qiwei Chen, Wei Li, and Dik Lun Lee. 2019. Multi-interest network with dynamic routing for recommendation at Tmall. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 2615–2623.
- [25] Jiacheng Li, Ming Wang, Jin Li, Jinmiao Fu, Xin Shen, Jingbo Shang, and Julian McAuley. 2023. Text is all you need: Learning language representations for sequential recommendation. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1258–1267.
- [26] Duncan McElfresh, Sujay Khandagale, Jonathan Valverde, Vishak Prasad C, Ganesh Ramakrishnan, Micah Goldblum, and Colin White. 2023. When do neural nets outperform boosted trees on tabular data? *Advances in Neural Information Processing Systems* 36 (2023), 76336–76369.
- [27] Nikil Pancha, Andrew Zhai, Jure Leskovec, and Charles Rosenberg. 2022. Pinnerformer: Sequence modeling for user representation at pinterest. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 3702–3712.
- [28] Qi Pi, Weijie Bian, Guorui Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Practice on long sequential user behavior modeling for click-through rate prediction. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2671–2679.
- [29] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PMLR, 8748–8763.
- [30] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. 2023. Robust speech recognition via large-scale weak supervision. In *International conference on machine learning*. PMLR, 28492–28518.
- [31] Kaushik Rangadurai, Yiqun Liu, Siddarth Malreddy, Xiaoyi Liu, Piyush Maheshwari, Vishwanath Sangale, and Fedor Borisjuk. 2022. Nxtpost: User to post recommendations in facebook groups. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3792–3800.
- [32] David E Rumelhart, Geoffrey E Hinton, Ronald J Williams, et al. 1985. Learning internal representations by error propagation.
- [33] Rico Sennrich, Barry Haddow, and Alexandra Birch. 2015. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909* (2015).
- [34] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems* 37 (2024), 68658–68685.
- [35] Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* (2014).
- [36] Piotr Skalski, David Sutton, Stuart Burrell, Iker Perez, and Jason Wong. 2023. Towards a foundation purchasing model: Pretrained generative autoregression on transaction sequences. In *Proceedings of the Fourth ACM International Conference on AI in Finance*. 141–149.
- [37] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.
- [38] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [39] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [40] Ruoxi Wang, Rakesh Shivanna, Derek Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed Chi. 2021. Dcn v2: Improved deep & cross network and practical lessons for web-scale learning to rank systems. In *Proceedings of the web conference 2021*. 1785–1797.
- [41] Dongchao Yang, Jinchuan Tian, Xu Tan, Rongjie Huang, Songxiang Liu, Xuankai Chang, Jiatong Shi, Sheng Zhao, Jiang Bian, Xixin Wu, et al. 2023. Uniaudio: An audio foundation model toward universal audio generation. *arXiv preprint arXiv:2310.00704* (2023).
- [42] Guri Zabërgja, Arind Kadra, and Josif Grabocka. 2024. Tabular Data: Is Attention All You Need? *arXiv preprint arXiv:2402.03970* (2024).
- [43] Dong Zhang, Shimin Li, Xin Zhang, Jun Zhan, Pengyu Wang, Yaqian Zhou, and Xipeng Qiu. 2023. Speechgpt: Empowering large language models with intrinsic cross-modal conversational abilities. *arXiv preprint arXiv:2305.11000* (2023).